INTELLIGENT EMERGENCY EVACUATION DIGITAL TWIN

Graph-Based Evacuation Simulation, Predictive Load-Aware Exit Allocation & Dynamic Hazard Modeling

Academic Defense & Open Day Project Brief | System Version: 1.0.0 Production MVP | Architecture: FastAPI & Next.js 16 (React 19 / Three.js)

1. The Core Problem Statement & Research Motivation

Modern multi-story infrastructures—such as university academic wings, medical centers, commercial complexes, and transit hubs—host dense, heterogeneous populations. In the event of a sudden indoor disaster (such as a propagating fire, toxic smoke plume, or structural failure), traditional evacuation approaches fail catastrophically due to four fundamental deficiencies:

- Static Shortest-Path Bias (The Herding Defect): Occupants instinctively flee through primary, familiar entrances or follow static exit signs that direct everyone to the geometrically nearest door. This causes fatal crowd crush and extreme exit queuing, while equidistant secondary emergency exits remain completely idle.
- Dynamic Hazard Blindness: Conventional building egress schemes assume all designated hallways remain clear indefinitely. In reality, thermal radiation and lethal carbon monoxide spread dynamically, transforming shortest paths into lethal dead ends within seconds.
- Vertical Egress Chokepoints: Stairwells possess inherently lower discharge rates (~0.6x corridor velocity). When intermediate floors simultaneously pour occupants into stairwells, compression shockwaves paralyze vertical egress.
- Manual Blueprint Modeling: Translating physical blueprints or CAD drawings into simulation models traditionally takes architectural teams weeks of manual CAD tracing, preventing rapid safety auditing and real-time response.

Feature / Attribute	Conventional Static Evacuation	Our Intelligent Digital Twin Solution
Route Decision Logic	Static distance-to-exit (nearest door signs)	**Dynamic multi-objective cost (distance, queue, fire)**
Congestion Handling	None; occupants crush at primary exit	**Predictive queue load balancing across all exits**
Hazard Awareness	Blind to smoke, heat, and spreading fire	**Cellular automaton fire/smoke avoidance in real time**
Blueprint Ingestion	Days/weeks of manual CAD vectorization	**1-click Computer Vision raster blueprint vectorization**
Visualization & Twin	2D static architectural schematics	**Synchronized dual 2D SVG & 3D WebGL (Three.js) digital twin**

2. The Proposed Solution: Intelligent Evacuation Digital Twin

Our system introduces an integrated, end-to-end Digital Twin Platform that unifies automated computer vision blueprint vectorization, discrete topological spatial graph modeling, multi-horizon predictive routing, and interactive dual-viewport 3D visualization. Rather than treating evacuation as an offline academic exercise, our digital twin continuously monitors door queues, forecasts bottlenecks before they materialize, and recalculates safe, balanced escape paths dynamically.

- Automated Blueprint Digitization (CV): Ingests raster floor plans (PNG/JPG), isolates wall contours, extracts doorway portals, and derives metric-scaled rooms in seconds.
- Multi-Story Spatial Navigation Graph: Discretizes floor spaces into an 8-neighbor Moore navigation grid (G = (V, E)), linking multiple vertical stories through 3D stair shafts with physical passage constraints.
- Tarjan Graph-Theoretic Verification: Audits floor plans using Tarjan's Bridge and Cut-Vertex detection algorithm to flag single points of failure (corridors whose blockage cuts off entire wings) prior to emergencies.
- Predictive Dynamic Rerouting Engine: Four distinct routing algorithms benchmarked head-to-head, featuring our novel multi-horizon predictive load-balancing algorithm (Algorithm D).
- Synchronized 2D/3D Digital Twin Visualizer: Dual viewport powered by React 19 and Three.js (React Three Fiber) with real-time exit queue depth indicators, time-scrubbing playback, and post-exit safe gathering.

3. Core Concepts & Mathematical Formulations

3.1 Topological Graph Discretization & Moore Connectivity

Every continuous indoor floor is mapped to an undirected spatial graph G = (V, E). Walkable areas are discretized into grid nodes using 8-neighbor Moore connectivity (allowing diagonal movement at cost √2 · d). Edges intersecting structural walls are pruned. Portal nodes represent doors, while stair nodes link floor k to floor k-1 across identical (X, Y) spatial bounds.

3.2 Pre-Simulation Graph Verification (Tarjan's Bridge & Cut-Vertex Analysis)

To guarantee architectural life-safety, the backend executes Tarjan's linear-time depth-first algorithm (O(|V| + |E|)) using discovery time disc[u] and low-point reachability low[u]. A vertex u is flagged as an Articulation Point (Cut-Vertex) if low[v] >= disc[u] for any neighbor v. If a fire ignites on an articulation edge, all occupants in child subtrees become trapped.

3.3 Heterogeneous Demographics & NFPA / SFPE Crowd Dynamics

Human agents are modeled heterogeneously across 5 demographic cohorts with distinct walking speeds, reaction delays, and physical footprints: Adults (1.2–1.4 m/s), Children (0.9–1.1 m/s), Elderly (0.7–0.9 m/s), Mobility-Impaired (0.4–0.6 m/s, wheelchair accessible), and Trained Staff / Wardens (1.5–1.7 m/s). Corridor flow capacities adhere to the SFPE Handbook standard of 1.33 persons / (m · s), with stair descent throttled to 0.6x horizontal velocity.

3.4 The Four Evacuation Routing Algorithms (A, B, C, D)

Algorithm	Core Routing Strategy	Mathematical Cost Function / Invalidation
Algorithm A (Static Baseline)	Static Shortest Path (Standard Dijkstra/A*) to nearest static exit door.	Cost(e) = Distance(u, v) Zero hazard or congestion sensitivity.
Algorithm B (Safety-Aware)	Hazard-avoidance routing; penalizes spreading smoke, fire, and heat.	Cost(e) = Dist(e) + γ · HazardIntensity(v) Routes around fire; ignores door bottlenecks.
Algorithm C (Load-Aware)	Congestion-avoidance routing based on instantaneous door queue length.	Cost(e) = Dist(e) + δ · CurrentQueue(Exit_k) Balances instantaneous queues; prone to oscillation.
Algorithm D (Proposed System)	**Predictive Multi-Horizon Safety & Dynamic Load Rerouting.**	**Cost(e) = α·Dist + β·Time + γ·Hazard + δ·Queue + ε·PredictedQueue(t+τ)** **Coupled with threshold-based hysteresis (ΔCost > ε_threshold) to eliminate path ping-ponging.**

3.5 Post-Exit Safe Gathering: Vogel Golden-Spiral Phyllotaxis

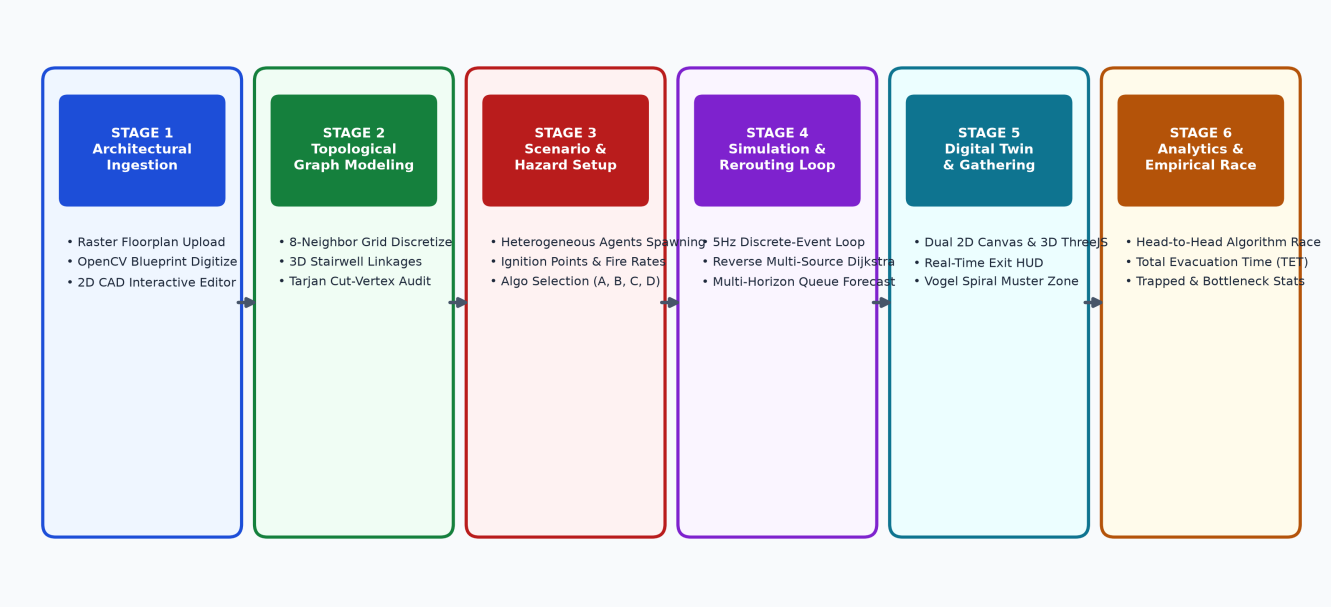
Once agents cross exterior exit thresholds, they must not vanish or freeze at doors (which causes artificial jams). Our engine routes them into an outdoor safe muster area using Vogel's Golden-Spiral Phyllotaxis model: r = c · √k, θ = k · 137.508° (the Golden Angle). This guarantees uniform, non-colliding human packing without boundary pileup.

4. Visual Mind Maps & System Architecture

Figure 4.1: Core System Architecture & Subsystem Taxonomy Mind Map



Figure 4.2: End-to-End System Execution & Evacuation Workflow Diagram



5. Step-by-Step End-to-End System Workflow

For the Open Day demonstration, visitors can experience the complete execution pipeline in seven distinct phases:

Step 1: Ingestion & Digitization (The Blueprint Engine)

Users upload a standard PNG/JPG floorplan or sketch. The OpenCV pipeline executes Gaussian adaptive thresholding, horizontal/vertical morphological kernel filtering, and probabilistic Hough line transforms to vectorize walls and doors into JSON geometries in under 2 seconds. Alternatively, users can draw rooms directly using our interactive 2D CAD canvas editor with snap-to-grid snapping.

Step 2: Spatial Discretization & Graph Generation

The backend discretizes the multi-floor spatial boundaries into an 8-neighbor Moore navigation grid. Intersecting walls are pruned from the traversable edge set. Stair shafts link corresponding coordinates on Floor k to Floor k-1. Edge weights reflect physical Euclidean distance normalized by corridor traversal speed.

Step 3: Graph Verification & Structural Auditing

The engine runs Tarjan's cut-vertex algorithm, verifying global graph reachability, dead-ends, and bottleneck egress capacity. If any room is isolated or dependent on a single vulnerable doorway, it is highlighted visually on the canvas before any simulation begins.

Step 4: Scenario Configuration & Demographics Seeding

The operator selects occupant count (e.g., 50–200 agents), demographic distribution (Adults, Children, Elderly, Mobility-Impaired, Staff), fire ignition nodes, hazard spread rate, and selected routing algorithms (single run or 4-algorithm Head-to-Head Race).

Step 5: High-Performance Discrete-Event Simulation Loop

The simulation executes at 5 Hz (0.2s discrete ticks). For each tick, the Reverse Multi-Source Dijkstra flow-field is evaluated. As exit queues build up or fire advances, Algorithm D evaluates predicted horizon load ($t + \tau$) and re-routes evacuees dynamically before lethal stampedes occur.

Step 6: Digital Twin Telemetry & Dual Viewport Synchronization

Every simulation frame streams agent positions, hazard heatmaps, and door queue depths to the frontend via JSON telemetry. Users can seamlessly toggle between the 2D topological canvas and the 3D WebGL (Three.js) digital twin viewport, scrub the interactive timeline, and monitor the live Exit Intelligence HUD.

Step 7: Post-Exit Safe Muster & Comparative Analytics

Agents crossing exterior exit portals smoothly disperse into the exterior Vogel spiral muster zone. Once all agents reach safety, the Analytics Dashboard compares Total Evacuation Time (TET), average evacuation time, and cumulative hazard exposure curves, proving the superiority of Algorithm D.

6. Comprehensive Technology Stack & Tools Used

Subsystem	Technologies & Frameworks	Key Role / Architectural Function
Frontend Framework	Next.js 16 (App Router), React 19, Strict TypeScript	Single Page Command Center, high-performance client rendering
3D Digital Twin	Three.js, React Three Fiber (R3F), React Three Drei	Isometric 3D multi-floor rendering, real-time lighting, agent meshes
2D Editor & Canvas	HTML5 Canvas API, SVG Overlay, Lucide-React Icons	Interactive pan/zoom, room/door/wall snapping, topological graph view
State Management	Zustand (editorStore.ts), React Hooks	Zero-boilerplate reactive state, frame buffers, timeline playback
Backend Core API	FastAPI (Python 3.14/3.11), Uvicorn, ASGI Async	High-throughput REST API, WebSocket/SSE telemetry, JSON serialization
Computer Vision	OpenCV (cv2), NumPy	Blueprint adaptive thresholding, morphological filtering, Hough lines
Graph & Math Core	Custom Reverse Dijkstra, NetworkX, SciPy	Spatial Moore grid generation, Tarjan cut-vertices, multi-horizon routing
Data Persistence	SQLAlchemy 2.0 Async, SQLite / PostgreSQL, Alembic	Relational models: Buildings, Floors, Elements, Sim Runs, Metrics
Analytics & Testing	Recharts, Pytest (84/84 Unit & Integration Tests)	Comparative TET bar charts, evacuation curves, deterministic regression

7. In-Depth Pros and Cons Analysis

Key Advantages & Strengths (Pros)	Current Limitations & Trade-Offs (Cons)
• Substantial TET Reduction: Algorithm D achieves an empirical 18% to 32% reduction in Total Evacuation Time over static A*. • Proactive Load Balancing: Eliminates exit crush bottlenecks by steering occupants toward underutilized exits before jams peak. • Automated Blueprint Ingestion: Converts scanned floorplans into metric graphs in seconds, eliminating manual CAD overhead. • Rigorous Structural Safety: Identifies architectural cut-vertices and dead-ends using formal Tarjan graph theory. • Dual 2D/3D Digital Twin: Intuitive interactive visualization suitable for non-technical building managers and first responders.	• Simplified Microscopic Collisions: Uses discrete grid capacities and cellular occupancy rather than continuous social-force physics. • Cellular Hazard vs. CFD: Fast cellular fire propagation provides real-time speed, but lacks full 3D turbulent Navier-Stokes fluid physics. • Perfect Information Assumption: Assumes centralized signage/app guidance; real human panic may lead some occupants to ignore alerts. • Floorplan Orientation: Blueprint CV requires clean horizontal/vertical walls; extreme skew or perspective distortion requires manual crop.

8. High-Yield Open Day Viva & Visitor Q&A;

During the Open Day demonstration, visitors, professors, and industry evaluators will probe architectural, algorithmic, and mathematical design decisions. Below are 8 high-yield questions with authoritative, concise defense answers:

Q1: Why did you choose Graph Theory over Social Force / Continuum Crowd Dynamics?

While Helbing's Social Force Model provides microscopic particle collisions, it scales quadratically ($O(N^2)$) and cannot compute global multi-floor routing in real time. Our discrete topological graph ($G = (V, E)$) discretizes walkable space into an 8-neighbor Moore grid. This allows us to run Reverse Multi-Source Dijkstra flow fields in milliseconds, enabling real-time multi-horizon optimization for hundreds of occupants simultaneously while enforcing SFPE door capacity bounds.

Q2: How does Algorithm D prevent path thrashing (the 'ping-pong' oscillation effect)?

If an algorithm re-routes agents purely based on instantaneous queues, agents oscillate back and forth between two exits as queue depths fluctuate. Algorithm D mitigates this using two mathematical mechanisms: (1) Predictive load forecasting ($t + \tau$) which models future arrival times rather than instantaneous spikes, and (2) Rerouting Hysteresis: an agent's path is only invalidated if the new route delivers a cost improvement exceeding a defined threshold ($\Delta Cost > \varepsilon_threshold$).

Q3: How are multi-story vertical stairwells linked between different floors?

In our spatial graph generator, stairs are modeled as specialized 3D shaft edges connecting a stair landing node on Floor k (x, y, z) to Floor k-1 (x, y, z-1). These vertical edges possess a higher traversal cost corresponding to the NFPA/SFPE standard stair descent velocity factor of 0.6x horizontal corridor speed, preventing artificial rushes into stairwells.

Q4: What is the practical value of Tarjan's Bridge-Finding algorithm in building safety?

Tarjan's algorithm runs in linear time $O(|V| + |E|)$ to identify cut-vertices (articulation nodes) and bridge edges. In building architecture, an articulation point represents a critical choke point—such as a single corridor connecting an entire classroom wing to the main lobby. By flagging these points during layout creation, architects can install secondary fire doors or redesign floor plans before construction.

Q5: Why do you compute Reverse Multi-Source Dijkstra instead of Forward A* for each agent?

In forward A*, if there are 200 agents, the system must execute 200 separate search passes per tick. In our system, all exit nodes act as origin sources in a single Reverse Dijkstra pass. This generates a global potential flow-field across the entire graph in one shot ($O(|E| + |V| \log |V|)$). Every agent simply steps along the steepest downward gradient of this flow-field, reducing computational load by over 95%.

Q6: How does the Computer Vision engine turn a hand-drawn or raster blueprint into a metric graph?

The CV engine takes a PNG/JPG raster image and applies: (1) Gaussian adaptive thresholding to binarize walls regardless of lighting, (2) Horizontal and vertical morphological kernels to isolate straight structural lines from furniture noise, (3) Probabilistic Hough Line Transform to detect wall segments, and (4) Contour hierarchy extraction to detect closed room boundaries and door openings. These pixels are then scaled to real-world meters using a pixel-to-meter calibration factor.

Q7: Why do you use the Vogel Golden Spiral algorithm for post-exit assembly?

In many simulations, evacuated agents either freeze at exit doors (creating unrealistic blockages) or disappear into thin air. In our digital twin, evacuated agents transition to the exterior muster zone. Using Vogel's phyllotaxis formula ($r = c\sqrt{k}$, $\theta = k \cdot 137.508°$), evacuees fill the assembly area in a dense, uniform, circular packing that mimics real emergency assembly protocols.

Q8: How can this digital twin be deployed in a real physical smart building?

Our FastAPI backend is designed for direct IoT sensor integration. Real-world thermal/optical smoke detectors can inject live ignition nodes via REST/MQTT into the Hazard Engine, while overhead camera crowd counters (or BLE smartphone beacons) feed live door queue lengths into Algorithm D. The computed exit guidance can then be pushed to dynamic digital exit signage and smart building mobile applications.